Hashmaps And Linked Lists

Hash Map Pattern

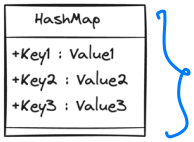
Definition:

Stores key-value pairs,
enabling fast data retrieval by directly accessing values through unique keys

Use Cases:

Fast data access and retrieval.
Storing relationships between pairs of data.
Implementing dictionaries, database indexing, caches.

Example:



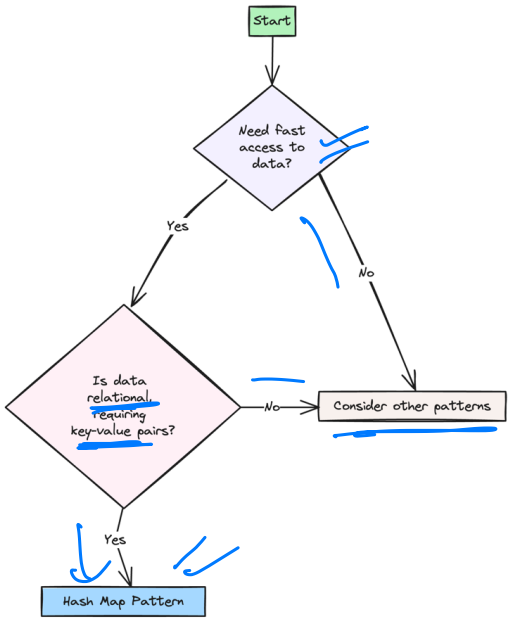
Advantages:

Efficient Data Access: O(1) time complexity for lookup, insert, and delete operations on average.

Flexibility in Key Usage: Unlike arrays, keys in hash maps can be of various data types

Avoids Data Duplication

Does My Problem Match This Pattern?




```
{
    key1 : value1,
    key2 : value2,
    ⋮
}
```

PROBLEM - 1

347. Top K Frequent Elements

Solved

Medium | Topics | Companies

Given an integer array `nums` and an integer `k`, return *the* `k` *most frequent elements*. You may return the answer in **any order**.

Example 1:

> **Input:** `nums = [1,1,1,2,2,3], k = 2`
>
> **Output:** `[1,2]`

Example 2:

> **Input:** `nums = [1], k = 1`
>
> **Output:** `[1]`

Example 3:

> **Input:** `nums = [1,2,1,2,1,2,3,1,3,2], k = 2`
>
> **Output:** `[1,2]`

Constraints:

- `1 <= nums.length <= 10^5`
- `-10^4 <= nums[i] <= 10^4`
- `k` is in the range `[1, the number of unique elements in the array]`.
- It is **guaranteed** that the answer is **unique**.

Brute:

G,

nums = [1, 1, 1, 2, 2, 3], K = 2

1: 3 times
2: 2 times
3: 1 time

[1, 2]

{1: 3, 2: 2, 3: 1}

[(1, 3), (2, 2), (3, 1)]

[1, 2, 1, 2, 1, 2, 3, 1, 3, 2], K = 2

{1: 4, 2: 4, 3: 2}

[(1, 4), (2, 4), (3, 2)]

ans = [1, 2]

TC = O(n) + O(n log(n))
SC = O(n)

Better: Based on heaps.

Optimal:

nums = [1, 1, 1, 2, 2, 3]

count = {1: 3, 2: 2, 3: 1}

K = 2

0	1	2	3	4	5
	[3]	[2]	[3]		

(n+1)

[3, 2]

[1, 2, 1, 2, 1, 2, 3, 1, 3, 2]

{1: 4, 2: 4, 3: 2}

(n+1)

0	1	2	3	4	5	6	7	8	9	10

[3] [1, 2]

K = 2

[1, 2]

$$\left.\begin{aligned} TC &= \underline{O(n) + O(n)} \\ SC &= \underline{O(n)} \end{aligned}\right\}$$

Linked Lists Patterns

Fast & Slow Pointers

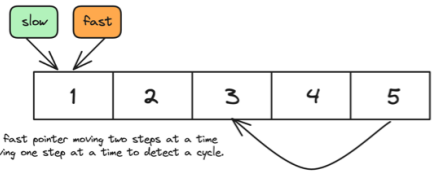
Definition:

uses two pointers traversing an iterable data structure at different speeds to identify distinguishable features

Use Cases:

Detecting cycles in a linked list or sequence.
Finding the middle element of a linked list.
Determining if a list is palindromic.
Finding the meeting point in a cycle.

Example:



In a linked list, use a fast pointer moving two steps at a time and a slow pointer moving one step at a time to detect a cycle.

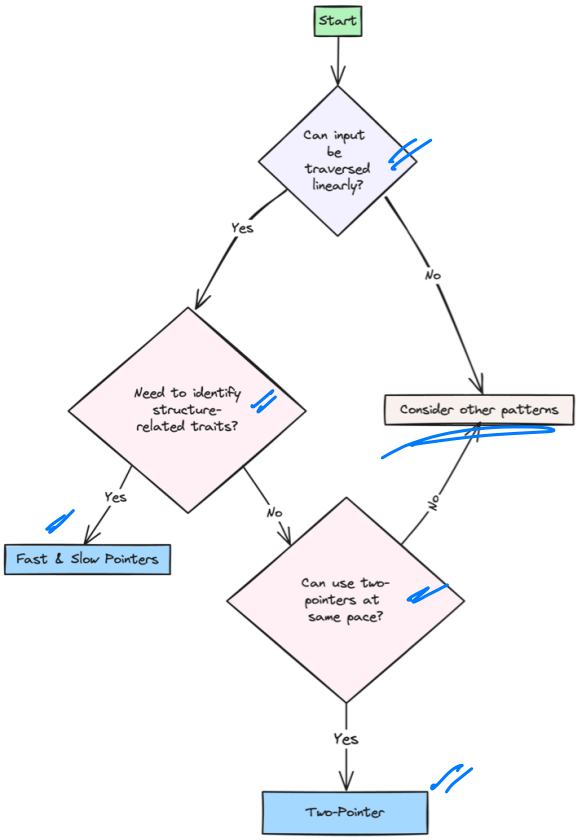
Advantages:

Efficient for cycle detection without extra space.

Useful for finding specific elements (like the middle) in a single pass.

Requires minimal extra space (O(1) space complexity)

Does My Problem Match This Pattern?



In-place Reversal of a Linked List

Definition:

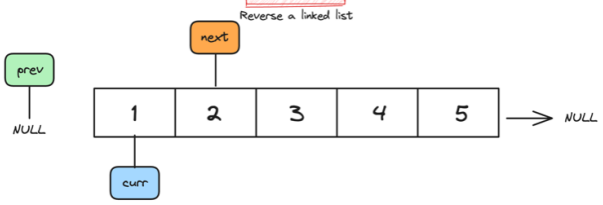
Reorganizes the nodes of a linked list in the opposite order directly, without additional memory.

Use Cases:

Reversing an entire singly or doubly linked list.
Reversing sub-lists within a linked list for pattern matching or reordering.
Used as a step in more complex algorithms, like palindrome checks.

Example:

Reverse a linked list

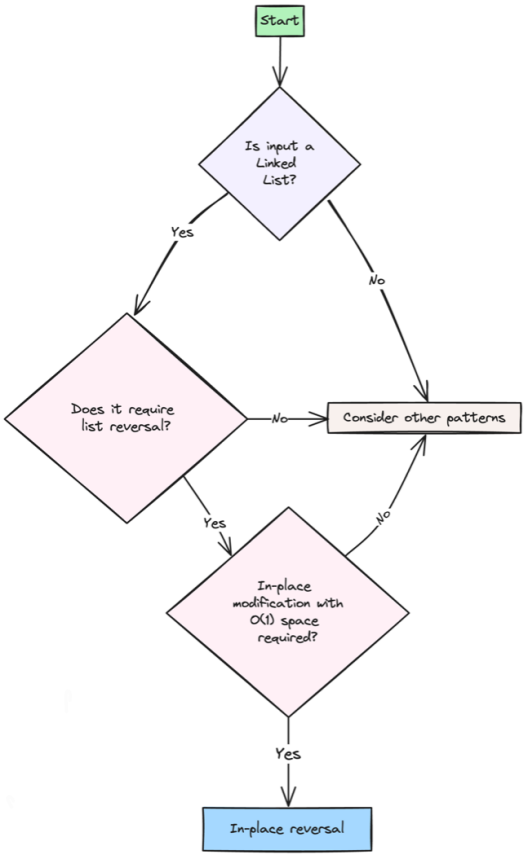


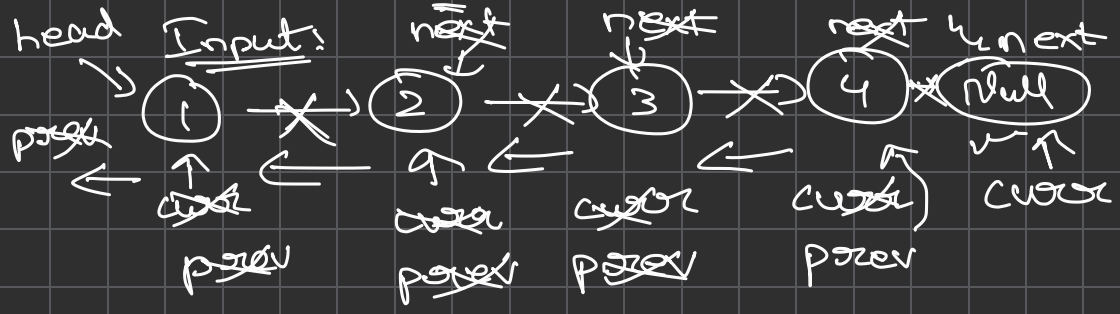
Advantages:

Space Efficiency: Utilizes O(1) space, not requiring extra lists or arrays.

Time Complexity: Achieves O(n) time efficiency by reorienting pointers in a single pass.

Does My Problem Match This Pattern?






```
prev = null, curr = head
while (curr != null)
{
    next = curr.next
    curr.next = prev
    prev = curr
    curr = next
}
return prev;
```

```
Node {
    val: int,
    next: Node
}
```

TC: O(n)
SC: O(1)

PROBLEM- 2

2095. Delete the Middle Node of a Linked List

Solved

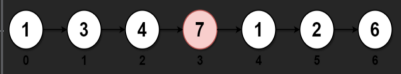
Medium | Topics | Companies | Hint

You are given the `head` of a linked list. **Delete** the **middle node**, and return *the* `head` *of the modified linked list*.

The **middle node** of a linked list of size `n` is the `⌊n / 2⌋th` node from the **start** using **0-based indexing**, where `⌊x⌋` denotes the largest integer less than or equal to `x`.

- For `n` = `1`, `2`, `3`, `4`, and `5`, the middle nodes are `0`, `1`, `1`, `2`, and `2`, respectively.

Example 1:

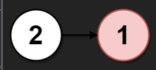



```
Input: head = [1,3,4,7,1,2,6]
Output: [1,3,4,1,2,6]
Explanation:
The above figure represents the given linked list. The indices of the nodes are written below.
Since n = 7, node 3 with value 7 is the middle node, which is marked in red.
We return the new list after removing this node.
```

Example 2:

```
Input: head = [1,2,3,4]
Output: [1,2,4]
Explanation:
The above figure represents the given linked list.
For n = 4, node 2 with value 3 is the middle node, which is marked in red.
```

Example 3:

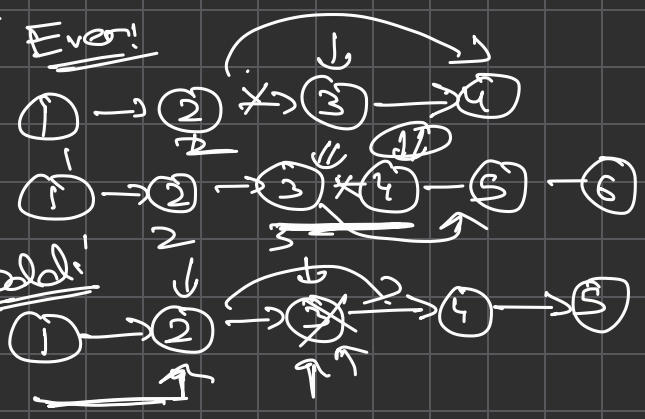



```
Input: head = [2,1]
Output: [2]
Explanation:
The above figure represents the given linked list.
For n = 2, node 1 with value 1 is the middle node, which is marked in red.
Node 0 with value 2 is the only node remaining after removing node 1.
```

Constraints:

- The number of nodes in the list is in the range `[1, 10⁵]`.
- `1 <= Node.val <= 10⁵`

Even:



$n = 5$

$5//2 = 2$

$n = 4$, $4//2 = 2$

$n = 6$, $6//2 = 3$

$\underline{curr.next} = \underbrace{curr.next}_{mid}.next$

$T.C = O(n) + O(n/2)$

$S.C = O(1)$

Node {
val : int
next : Node
}

Pseudocode:

Brute force

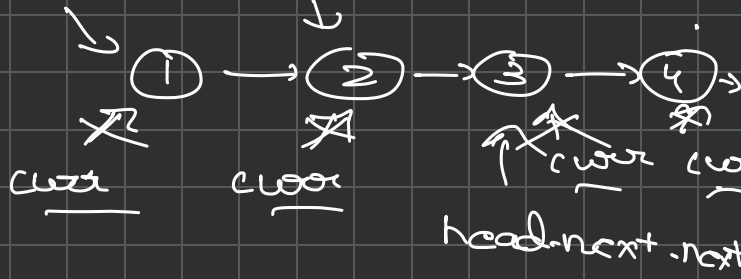
#① Find length

```
n = 0
curr = head
while (curr != None)
{
    n = n + 1
    curr = curr.next
}
```

#②

```
mid_prev = n//2
curr = head
```

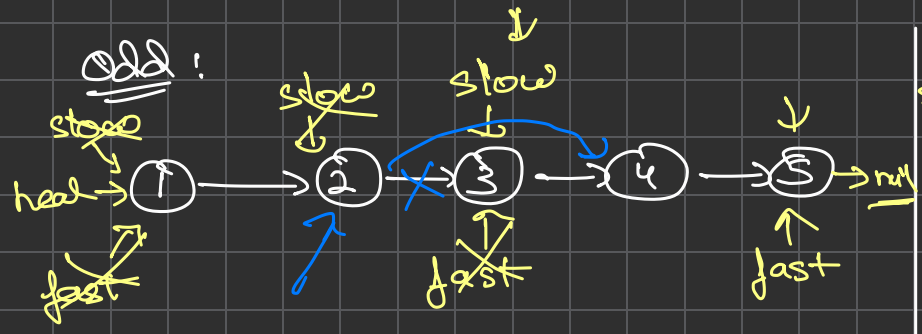
head.next



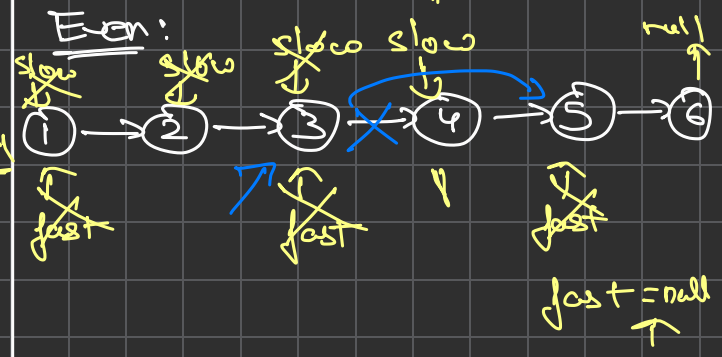
```
while (curr != None)
{     if (mid == 0) { curr.next = curr.next.next;
                      break }
      mid = mid - 1
      curr = curr.next
}
return head;
```

Optimal: Slow - fast pointer approach ⇓ for finding middle node

Odd:



Even:

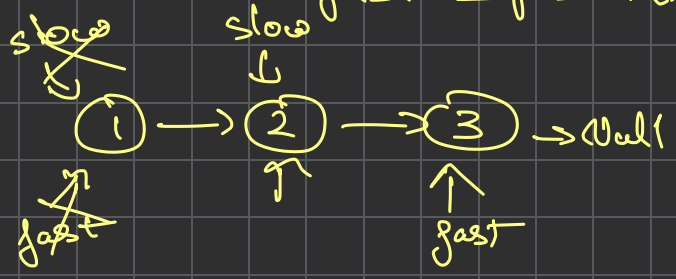


slow = slow.next

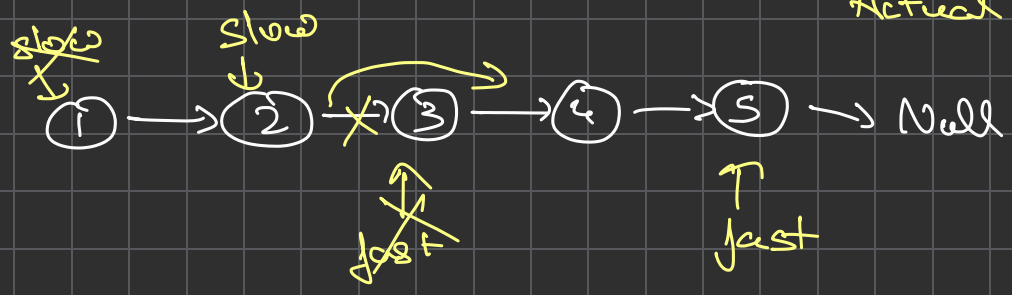
fast = fast.next.next

TC = $O(n/2)$

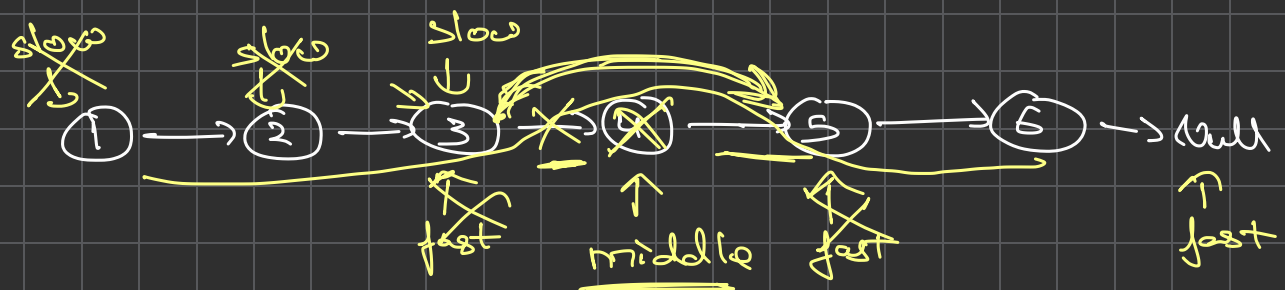
SC = $O(1)$

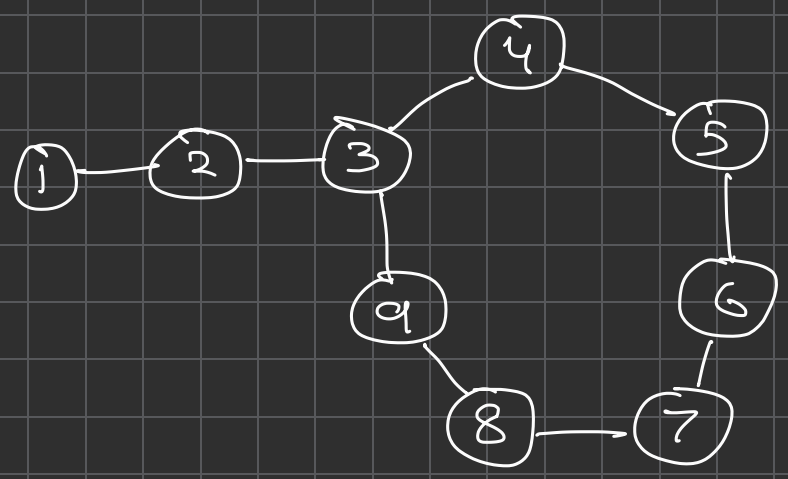


⇓ Actual approach



⇓ slow.next = slow.next.next




1
2
3
4
5
6
7
8
9

PROBLEM - 3

141. Linked List Cycle

Solved

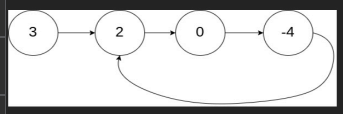
Easy | Topics | Companies

Given `head`, the head of a linked list, determine if the linked list has a cycle in it.

There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the `next` pointer. Internally, `pos` is used to denote the index of the node that tail's `next` pointer is connected to. **Note that `pos` is not passed as a parameter**.

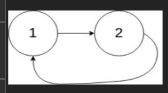
Return `true` *if there is a cycle in the linked list*. Otherwise, return `false`.

Example 1:




```
Input: head = [3,2,0,-4], pos = 1
Output: true
Explanation: There is a cycle in the linked list, where the tail connects to the 1st node (0-indexed).
```

Example 2:



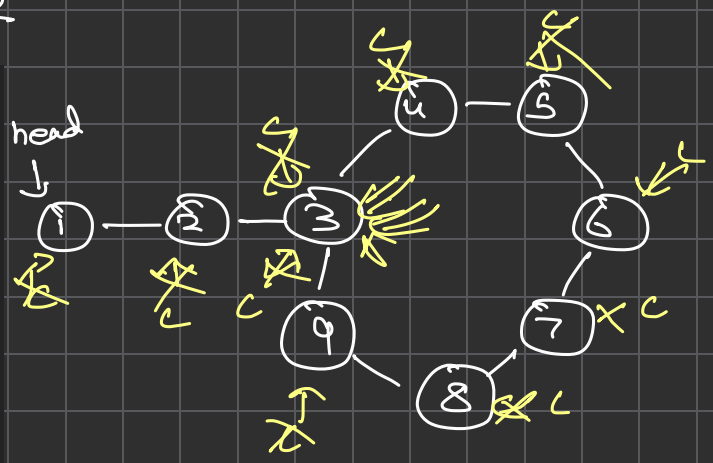

```
Input: head = [1,2], pos = 0
Output: true
Explanation: There is a cycle in the linked list, where the tail connects to the 0th node.
```

Example 3:

```
Input: head = [1], pos = -1
Output: false
Explanation: There is no cycle in the linked list.
```

Constraints:

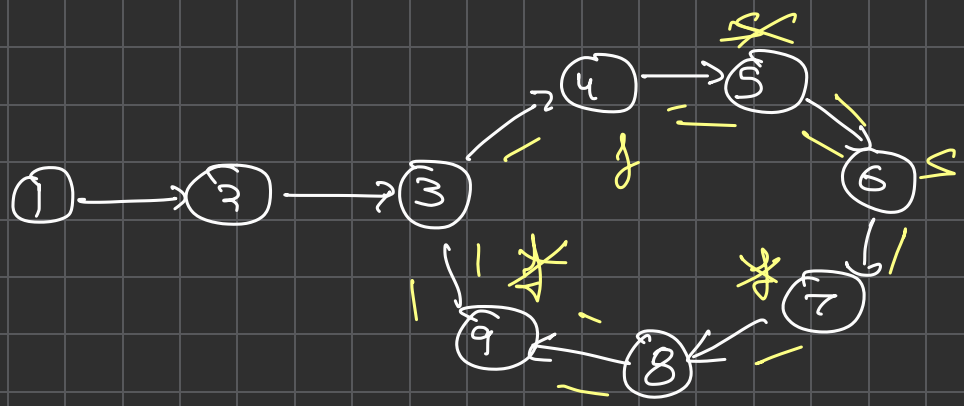
- The number of the nodes in the list is in the range `[0, 10⁴]`.
- `-10⁵ <= Node.val <= 10⁵`
- `pos` is `-1` or a **valid index** in the linked-list.



1 → True
2 → True
3 → True

1 → 2 → 3 → Null

T.C = O(n)
S.C = O(n)



d
d − 1
d − 2
d − 3
d − d = 0

d = 5
d = 4
d = 3
d = 2
d = 0

5 − − 0

PROBLEM - 4

49. Group Anagrams

Medium | Topics | Companies

Given an array of strings `strs`, group the anagrams together. You can return the answer in **any order**.

Example 1:

> **Input:** `strs = ["eat","tea","tan","ate","nat","bat"]`
>
> **Output:** `[["bat"],["nat","tan"],["ate","eat","tea"]]`
>
> **Explanation:**
>
> - There is no string in strs that can be rearranged to form `"bat"`.
> - The strings `"nat"` and `"tan"` are anagrams as they can be rearranged to form each other.
> - The strings `"ate"`, `"eat"`, and `"tea"` are anagrams as they can be rearranged to form each other.

Example 2:

> **Input:** `strs = [""]`
>
> **Output:** `[[""]]`

Example 3:

> **Input:** `strs = ["a"]`
>
> **Output:** `[["a"]]`

=> Anagram: It is a word or phrase formed by re-arranging the letters of a different word or phrase, using all the original chars. exactly once.

eat -> aet
ate -> aet
tea -> aet

tan -> ant
nat -> ant

ttan -> antt
natt -> antt

{ sorted_chars: [- - -] }

K avg. len of string

TC = O(n.(K log K))
S.C = O(n)

a b c d e - - - - - , z
0, 1, 2, 3, 4 - - - - - - , 26

arr = [| | |]

{ eat, ate, tea }